Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 05

React: stare, evenimente și formulare

Facem interfața să „țină minte”: un formular de contact controlat, cu validare în client, și o căutare care filtrează pe loc lista de produse.

Tema	Tema 6 din planul disciplinei (examene.pe.ub.ro), evenimente și formulare, aplicată în React
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` pornit, cu laboratorul 04 terminat (pagina `/despre`)
La final aveți	pagina `/contact` cu formular validat și căutarea pe pagina de produse, salvate pe GitHub

1. Obiective

La finalul laboratorului veți putea:

- păstra date care se schimbă într-o componentă, cu `useState`;
- trata evenimentele `onClick`, `onChange` și `onSubmit` în React;
- scrie un formular controlat, cu mesaje de eroare clare lângă câmpuri;
- explica rolul lui `useEffect` la încărcarea datelor de la API;
- filtra o listă în client, în timp ce utilizatorul tastează.

2. Pe scurt: starea și formularele controlate

Props-urile vin din afară și nu se modifică. Datele care se schimbă în timp ce utilizatorul lucrează (ce a tastat, dacă a trimis formularul) se păstrează în **stare** (*state*) [3]. `const [valoare, setValoare] = useState(inițial)` creează o variabilă de stare și funcția care o schimbă. La fiecare apel `setValoare(...)`, React redesenează componenta cu noua valoare. Exemplul minim, un buton care numără click-urile:

```
const [count, setCount] = useState(0);
// ...în JSX:
<button type="button" onClick={() => setCount(count + 1)}>
  Ați apăsat de {count} ori
</button>
```

Un formular este **controlat** când valoarea fiecărui câmp vine din stare (`value={...}`) și orice tastare actualizează starea (`onChange`). Astfel, datele sunt mereu disponibile în JavaScript, pentru validare sau trimitere. Validarea din client ajută utilizatorul, dar nu înlocuiește verificarea de pe server, pe care o facem în laboratoarele de backend [1], [4].

3. Pornirea

1. Porniți **Docker Desktop**. Deschideți folderul `daw-template-ti-info-2026-2027` în editor și terminalul integrat.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

4. Pagina de contact, cu validare

Unde. În `frontend/app/src/pages/` creați fișierul `ContactPage.jsx`. Îl scriem în două părți, una sub alta, în același fișier.

Partea 1 – regulile de validare și starea formularului:

```
import { useState } from 'react';

const EMPTY = { name: '', email: '', message: '' };

function validate(values) {
  const errors = {};
  if (values.name.trim().length < 3) {
    errors.name = 'Numele trebuie să aibă cel puțin 3 caractere.';
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(values.email.trim())) {
    errors.email = 'Introduceți o adresă validă, de exemplu ion@exemplu.ro.';
  }
  if (values.message.trim().length < 10) {
    errors.message = 'Mesajul trebuie să aibă cel puțin 10 caractere.';
  }
  return errors;
}
```

Partea 2 – componenta: funcțiile pentru evenimente și formularul:

```
export default function ContactPage() {
  const [values, setValues] = useState(EMPTY);
  const [touched, setTouched] = useState({});
  const [sent, setSent] = useState(false);

  const errors = validate(values);
  const isValid = Object.keys(errors).length === 0;
  function handleChange(event) {
    const { name, value } = event.target;
    setValues({ ...values, [name]: value });
    setSent(false);
  }
  function handleBlur(event) {
    setTouched({ ...touched, [event.target.name]: true });
  }
  function handleSubmit(event) {
    event.preventDefault();
    if (!isValid) return;
    console.log('Mesaj de trimis:', values);
    setSent(true);
    setValues(EMPTY);
    setTouched({});
  }
```

În continuare, tot în funcția `ContactPage`, JSX-ul formularului:

```
  return (
    <section>
      <h1>Contact</h1>
      {sent && <p className="form-success">Mesajul a fost trimis. Mulțumim!</p>}
      <form onSubmit={handleSubmit} noValidate>
        <div className="form-field">
          <label htmlFor="name">Nume</label>
          <input id="name" name="name" value={values.name}
            onChange={handleChange} onBlur={handleBlur} />
          {touched.name && errors.name && (
            <p className="form-error">{errors.name}</p>
          )}
        </div>
```

Continuarea formularului, cu celelalte două câmpuri și butonul:

```
        <div className="form-field">
          <label htmlFor="email">E-mail</label>
          <input id="email" name="email" type="email"
            value={values.email}
            onChange={handleChange} onBlur={handleBlur} />
          {touched.email && errors.email && (
            <p className="form-error">{errors.email}</p>
          )}
        </div>
        <div className="form-field">
          <label htmlFor="message">Mesaj</label>
          <textarea id="message" name="message" rows="5"
            value={values.message}
            onChange={handleChange} onBlur={handleBlur} />
          {touched.message && errors.message && (
            <p className="form-error">{errors.message}</p>
          )}
        </div>
        <button type="submit" disabled={!isValid}>Trimite</button>
      </form>
    </section>
  );
}
```

1. În `App.jsx`, lângă celelalte importuri, adăugați `import ContactPage from './pages/ContactPage';`, iar lângă celelalte linii `<Route ...>`, adăugați `<Route path="/contact" element={<ContactPage />} />`.
2. În meniu, după modelul legăturilor existente (inclusiv cea către `/despre`), adăugați legătura **Contact** către `/contact`.
3. La sfârșitul lui `App.css`, adăugați:

```
.form-field {
  display: flex; flex-direction: column; gap: 0.25rem;
  margin-bottom: 1rem; max-width: 480px;
}
.form-error { color: #b42318; margin: 0; font-size: 0.9rem; }
.form-success { color: #2e7d32; font-weight: bold; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
```

Rezultat. Pe http://localhost:5173/contact, butonul **Trimite** este estompat cât timp un câmp nu respectă regula. Eroarea unui câmp apare după ce îl părăsiți (`onBlur`), nu din prima literă. Cu toate câmpurile corecte, butonul se activează; la trimitere apare mesajul verde, formularul se golește, iar datele apar în consolă.

De ce. `name="..."` al fiecărui câmp coincide cu cheia din `values`, așa că o singură funcție, `handleChange`, servește toate câmpurile. `{ ...values, [name]: value }` copiază obiectul și schimbă doar câmpul tastat: în React, starea nu se modifică direct, ci se înlocuiește.

5. useEffect și căutarea pe pagina de produse

Deschideți `pages/ProductsPage.jsx`. Veți găsi un tipar asemănător cu acesta (numele pot diferi puțin):

```
const [products, setProducts] = useState([]);

useEffect(() => {
  getProducts().then(setProducts);
}, []);
```

`useEffect` rulează cod **după** afișarea componentei; aici cere produsele de la API, prin funcția din `api.js`. Lista goală `[]` de la final înseamnă „o singură dată, la prima afișare”. Fără ea, efectul ar rula după fiecare redesenare, iar fiecare răspuns ar declanșa o nouă cerere [3].

1. Sub linia cu `useState` a produselor, adăugați starea căutării și lista filtrată (dacă variabila nu se numește `products`, folosiți numele din fișier):

   ```
   const [search, setSearch] = useState('');
   const term = search.trim().toLowerCase();
   const visibleProducts = products.filter((product) =>
     product.name.toLowerCase().includes(term)
   );
   ```

2. În JSX, deasupra listei, adăugați câmpul de căutare:

   ```
   <input
     type="search"
     placeholder="Caută după nume…"
     aria-label="Caută produse"
     value={search}
     onChange={(event) => setSearch(event.target.value)}
   />
   ```

3. În locul în care lista este afișată, înlocuiți `products.map(` cu `visibleProducts.map(`. Restul rămâne neschimbat.
4. Sub listă, adăugați mesajul pentru căutările fără rezultat:

   ```
   {visibleProducts.length === 0 && (
     <p>Niciun produs nu corespunde căutării.</p>
   )}
   ```

Rezultat. Lista se restrânge la fiecare literă tastată, fără nicio cerere nouă către server: filtrarea se face în browser, pe datele deja primite. Căutarea nu ține cont de majuscule. Dacă nu aveți produse, adăugați câteva din Swagger (http://localhost:8080/api/doc, `POST /api/products`).

6. Exerciții

1. Adăugați pe pagina **Despre** butonul-contor din secțiunea 2 (nu uitați `import { useState } from 'react';`). Apoi faceți un al doilea buton, **Resetează**, care readuce contorul la 0.
2. Adăugați formularului de contact un câmp **Subiect**, obligatoriu, cu minimum 5 caractere și mesaj de eroare propriu.
3. Afișați sub câmpul **Mesaj** un contor de caractere: `{values.message.length} / 500`. Adăugați și regula „cel mult 500 de caractere”.
4. ★ (opțional) Pe pagina de produse, adăugați un buton **Șterge căutarea**, vizibil doar când `search` nu este gol.
5. ★ (opțional) Afișați câte produse corespund căutării: „3 din 10 produse”.

7. Salvarea lucrului și oprirea

```
git status
git add .
git commit -m "Laboratorul 05: formularul de contact si cautarea produselor"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

8. Ce arătați la final

- pagina `/contact`: butonul dezactivat, erorile lângă câmpuri, mesajul de succes;
- căutarea care filtrează produsele în timp ce tastați și explicația rolului lui `[]` din `useEffect`;
- exercițiile 1–3 rezolvate și commit-ul vizibil pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – React useState:** https://www.w3schools.com/react/react_usestate.asp
- **W3Schools – formulare în React:** https://www.w3schools.com/react/react_forms.asp
- **React, în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PLZuysB_DjFARX5kWZGu8hvBbmw-3VWd8w
- **React – starea unei componente (documentație):** https://react.dev/learn/state-a-components-memory
- **React – sincronizarea cu efecte:** https://react.dev/learn/synchronizing-with-effects

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[3] Meta Open Source, *React – Learn* (State: A Component’s Memory; Synchronizing with Effects). react.dev/learn

[4] W3Schools, *JavaScript Form Validation*. w3schools.com

[5] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.